

The Name of the Title Is Hope

Anonymous Author(s)

Abstract

LLM agents with web access are increasingly becoming the default for user-facing question answering. Reliable evaluation in this setting requires tests that remain unseen at train time and cannot be answered via verbatim lookup at inference. However, standard practice still relies on static benchmarks that age quickly and are vulnerable to contamination. We analyze two channels, pretraining leakage where test items appear in the pretraining data, and run-time leakage where the agent finds the exact target answer verbatim during evaluation and reports it rather than demonstrating genuine problem solving. We uncover evidence across five QA datasets for both pretraining and run-time leakage, and we show that static evaluation can overestimate capability. We introduce DynamicWebQA, a dynamic evaluation framework that constructs questions and verifiable answers at evaluation time from relevant web sources. The framework enforces multi-document grounding, records evidence chains, and offers controllable complexity through source set size, number of hops, and reasoning constraints. By generating fresh test instances, DynamicWebQA reduces memorization risk, mitigates run-time leakage, and naturally covers time-sensitive queries.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper;** *Generate the Correct Terms for Your Paper;* Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

Anonymous Author(s). 2018. The Name of the Title Is Hope. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, ?? pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Benchmark leakage refers to the contamination that occurs when a model has prior exposure to a benchmark’s test items during pretraining, leading to inflated evaluation scores and undermining the benchmark’s validity [? ?]. Since most LLMs are pretrained indiscriminately on publicly available web content, there is a high likelihood that they might have encountered benchmark datasets hosted on platforms like HuggingFace, GitHub, Kaggle, and similar

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

sites. If the base LLM used in a WebQA agent has already seen the test examples during pretraining, the evaluation effectively becomes a case of testing on the training set — severely compromising the credibility of the reported results.

Dynamic benchmarking is an emerging evaluation paradigm that generates or adapts test samples at evaluation time, so models are scored on fresh, unseen data and testset contamination is avoided [? ? ? ? ?]. That means the agent does not see the same test sample twice, effectively addressing issues such as LLM memorization.

In this work, we refer to information that can change over time, such as the stock price of Tesla, as evolving information. Arguably, this type of content could be labeled as dynamic content or temporal content, but that might cause confusion, since we are already using the term “dynamic” to describe an evaluation paradigm (dynamic benchmarking), and “temporal” might lead readers to associate it with time-series data. To avoid such confusion, we adopt the term evolving information to specifically denote factual content that may shift over time without implying a particular format or evaluation framework.

2 Related Work

Open-Domain QA (ODQA). Open-domain question answering studies how to find and use evidence from very large corpora to answer free-form questions. A sequence of datasets marked key shifts in task design: SQuAD [?] established span-based reading comprehension; Natural Questions [?] moved to real search queries; TriviaQA [?] stressed compositional, trivia-style questions with substantial lexical and syntactic variation; and HotpotQA [?] formalized multi-hop reasoning across multiple documents with sentence-level supporting facts and comparison questions, while including distractors to test *focus under noise*. Models such as fusion-in-decoder (FiD) [?] showed that aggregating many retrieved passages is a powerful way to synthesize scattered evidence.

Another important dimension in QA is *time*. RealTime QA [?] and FreshQA [?] evaluate whether systems can answer what is true *right now*, revealing failure modes when indices are stale or retrieved content is outdated or contradictory. These settings highlight needs that go beyond span extraction: recognizing when the answer has changed, when a question carries a false premise, and when recent evidence is required over older but higher-ranked documents. Finally, surface simplicity does not imply task simplicity. SimpleQA [?] demonstrates that short, seemingly straightforward questions can still require precise world knowledge or multi-step reasoning, indicating that difficulty is not well represented by length or phrasing.

Retrieval-Augmented Generation (RAG). Modern RAG approaches follow a common pipeline – given a query, retrieve top- k passages from a non-parametric memory and condition a generator on them. Foundational formulations include RAG [?], REALM [?], RETRO [?], and ATLAS [?], which together demonstrate that query-time retrieval improves knowledge-intensive tasks while enabling the external memory to be updated without re-training the parametric model. Recent surveys study components and variants of this paradigm [?].

In the literature, RAG is distinguished from broader tool-augmented generation – RAG requires *query-conditioned retrieval* whose contents directly condition the generator. Calculators, code execution, or static always-prepended context are typically treated as non-retrieval tools [?]. In practice, the retrieval index can be the open web or enterprise/local corpora, and the mechanism can be lexical (e.g., BM25) or dense; the generator then aggregates evidence across multiple passages [?].

Beyond single-shot RAG, agentic frameworks let models iteratively issue searches, follow links, and cite sources. WebGPT augments an LLM with browsing and requires collecting references during answering [?]. ReAct further couples reasoning traces with tool calls (e.g., search, calculator) in a unified prompting loop, enabling multi-hop planning and verification [?]. These approaches often embed RAG steps inside a broader plan-act-observe loop, allowing complex queries that require cross-document synthesis.

Data Contamination. Data contamination arises when evaluation items (or close paraphrases and answer-bearing spans) leak into the corpora used for pretraining, instruction tuning, or fine-tuning. In such cases, test behaviour can devolve into *lookup* of memorized strings rather than the intended inference or problem

solving, inflating reported performance and obscuring brittleness [?]. Prior studies connect leakage to memorization dynamics: larger models memorize more and earlier in training; duplication and longer prompts amplify verbatim regurgitation; and model capacity allows substantial unintended storage of datapoint-specific information [???]. At the same time, verbatim completion is not definitive evidence of membership—models can perfectly complete sequences never explicitly seen during training—so completion- or n -gram-based membership tests must be interpreted cautiously [?]. The measurable impact of contamination is further mediated by dataset construction, model scale, and properties of pretraining data [?].

[?] organize detection techniques by *data availability*. With candidate corpora available (*open-data*), methods compare evaluation items against sources via instance-/dataset-level span matching, embedding similarity, or paraphrase detection. When training data are unavailable (*closed-data*), researchers rely on indirect evidence such as performance analyses, membership-inference attacks, and instance-level probes of memorization and model confidence. Despite blind spots, approaches based on span matching remain the practical default in most foundational efforts [????] – as they are auditable and more importantly, tractable at web scale. Auditing a single question or sentence against a large crawl is a needle-in-a-haystack search; doing so for entire benchmarks across billions of documents renders more sophisticated but compute-heavy detectors infeasible. Moreover, for closed models without training-set access, behavioural signals alone rarely certify contamination [??].

Retrieval-augmented and agentic QA introduce a new concern – these systems may retrieve and quote target items from web or enterprise indices at *run time*, creating search-time data contamination that boosts scores without demonstrating reasoning [?]. [?] show that even small leakage rates (e.g., on the order of a few percent) can reorder competitive leaderboards once online access is enabled, undermining trust relative to purely offline evaluation.

Dynamic Benchmarking. Most benchmarks for agentic or retrieval-augmented QA remain static and publicly accessible (e.g., HotpotQA), which makes them susceptible to contamination and memorization [????]. As models and training corpora grow, leakage can inflate reported capability and mask brittleness. This has motivated *dynamic* evaluation paradigms that resist memorization and better reflect real-world, evolving information needs. Early platforms like Dynabench and Dynaboard supported human-and-model-in-the-loop data collection and hosted live evaluations [???]. Other works instantiate dynamic settings in specific environments (e.g., AndroidWorld for interactive agents) [?] and in time-sensitive QA streams.

TBD

3 Modeling Contamination in RAG-Based QA

Retrieval-Augmented Generation (RAG) systems combine a parametric model M with external sources retrieved at inference. This creates two leakage channels: (i) *pretraining contamination*, where an evaluation item (or a close paraphrase / answer-bearing span) appears in the corpus used to pretrain M ; and (ii) *retrieval-time contamination*, where the retriever surfaces such material during

evaluation, enabling lookup rather than reasoning. Our experiments instantiate web-enabled RAG (“web-RAG”), but the formalism below applies to RAG broadly.

3.1 Definitions

Let M be pretrained on corpus D_M . Let D_{eval} contain items $x = (q, a)$ with question q and answer a . Let $\mathcal{R}(q)$ be the set of documents returned by the retriever for query q . Following [?], let f be a similarity function and τ a threshold; $\tau=1$ corresponds to *strict/verbatim* contamination.

We study both *question-only* and *question+answer* leakage, noting that even input-only (question-only) leakage can shift measured performance [?]. We consider two views of an item:

$$\pi_{\text{in}}(q, a) = q \quad (\text{question-only, a.k.a. input-only}),$$

$$\pi_{\text{in+lab}}(q, a) = q \parallel a \quad (\text{question+answer, a.k.a. input+label}).$$

Contamination criterion. For a channel $C \in \{\text{pre}, \text{ret}\}$ and view $v \in \{\text{in}, \text{in+lab}\}$, we say D_{eval} is contaminated if

$$\exists x \in D_{\text{eval}}, \exists d \in \mathcal{S}_C(x) \text{ s.t. } f(\pi_v(x), d) \geq \tau, \quad (1)$$

with source sets

$$\mathcal{S}_{\text{pre}}(x) = D_M, \quad \mathcal{S}_{\text{ret}}(x) = \mathcal{R}(q).$$

When evidence must be explicitly *answer-bearing*, we additionally require a predicate $g(a, d) \in \{0, 1\}$ (e.g., answer mention or high-overlap), and record positives of

$$\exists x, d \text{ s.t. } f(q, d) \geq \tau_q \wedge g(a, d) = 1. \quad (2)$$

3.2 Implementation details

We instantiate f using five auditable open-data detectors frequently used in LLM pre-training studies and consolidated in [?]:

- **Exact Match (EM)** [?].
- **8-gram words** (GPT-2 pre-training study) [?].
- **13-gram words** (GPT-3 pre-training study) [?].
- **50-character substring** (GPT-4 technical report) [?].
- **8-gram words with 70% threshold** (PaLM pre-training study) [?].

Span-based detectors scale to long documents, handle the length mismatch between short questions and web pages, and yield concrete evidence spans. While semantic detectors (embeddings / phrase) may increase recall, they are harder to audit and become infeasible across disparate lengths; we therefore adopt span-based methods as primary signals, consistent with cautions in [?] and common audit practice [?].

Pretraining channel (candidate retrieval). Full-membership testing against all datapoints in web-scale pretraining corpora is infeasible. Following the candidate-set approximation in open-data audits [?], we index the CommonCrawl C4 corpus in an Elastic-compatible service (AWS OpenSearch). For each $x = (q, a)$, we retrieve top- k candidates with BM25 (default $k=1000$) and apply the detectors between π_{in} or $\pi_{\text{in+lab}}$ and each candidate document. Any positive flag indicates potential pretraining contamination.

Retrieval-time channel (web-RAG). For each q , we query SearxNG [?] with the raw question string, pool results from the first three result pages across underlying engines, and define $\mathcal{R}(q)$ accordingly. We (i) compute contamination using (??) and, when needed, (??) over this pool, and (ii) use the same pool as retrieval context for answer generation. This isolates run-time leakage introduced by search from memorization in M .

Reporting. For each dataset and channel we provide: (a) question-only and question+answer contamination rates; (b) per-detector breakdowns; and (c) overlap between pretraining and retrieval-time hits.

4 Method

We propose a method that utilizes *seed graphs* to anchor future question generation instead of relying on static datapoints that are susceptible to memorization or data contamination. Because each evaluation round renders a fresh QA pair from each graph, no fixed test item can be leaked ahead of time, yet all rounds remain semantically comparable via the common anchors.

4.1 Seed Graphs

At evaluation round t , we maintain a set $\mathcal{S}_t = \{G_1^{(t)}, \dots, G_{n_t}^{(t)}\}$ of seed graphs. Each graph encodes the evidence trail (e.g., queries, retrieved documents, answer node) for a distinct information need. To support evolving information, graphs are constructed at evaluation time.

4.2 QA Generation

The generation function Gen takes a seed graph $G_i^{(t)}$, a random seed $r_{i,t} \sim \mathcal{R}$, and a fixed generation configuration to produce one QA pair:

$$(q_{i,t}, a_{i,t}) = \text{Gen}(G_i^{(t)}; r_{i,t}, \theta) \quad (3)$$

Here, Gen is an LLM-based generator that bundles a frozen prompt template and fixed decoding settings (e.g., temperature, top- p , stop tokens). $r_{i,t}$ is an i.i.d. sample from randomness source $\mathcal{R}$, introducing per-sample stochasticity. Holding θ constant and varying only $r_{i,t}$ ensures that each system sees independent but identically distributed QA samples.

4.3 Distributional Consistency

Sampling an independent seed $r_{i,t} \sim \mathcal{R}$ for each graph yields the per-graph distribution:

$$\mathcal{P}_{i,t} = \text{Dist}(\text{Gen}(G_i^{(t)}; r, \theta)) \quad (4)$$

Because seeds are independent across graphs, the round- t dataset $D_t = \{(q_{1,t}, a_{1,t}), \dots, (q_{n_t,t}, a_{n_t,t})\}$ is an i.i.d. sample from the product measure:

$$\mathcal{P}_t = \prod_{i=1}^{n_t} \mathcal{P}_{i,t} \quad (5)$$

For a model M under evaluation, let $\text{score}((q, a), M)$ be any per-question metric (e.g., exact match, F1, log-loss). The aggregate dataset score is defined as:

$$\text{Agg}(D_t, M) = \frac{1}{n_t} \sum_{i=1}^{n_t} \text{score}((q_{i,t}, a_{i,t}), M) \quad (6)$$

By linearity of expectation and independence of $(q_{i,t}, a_{i,t}) \sim \mathcal{P}_{i,t}$:

$$\mathbb{E}_{D_t \sim \mathcal{P}_t} [\text{Agg}(D_t, M)] = \frac{1}{n_t} \sum_{i=1}^{n_t} \mathbb{E}_{(q,a) \sim \mathcal{P}_{i,t}} [\text{score}((q, a), M)] \quad (7)$$

This shows that the expected accuracy is simply the average of the per-graph expectations—so every seed graph contributes equally.

4.4 Collision Bound

Assume for every seed graph $G_i^{(t)}$, the generator can produce K distinct question–answer pairs:

$$C_{i,t} = \{(q, a) \in \mathcal{P}_{i,t} \mid \Pr((q, a)) > 0\}, \quad |C_{i,t}| = K$$

To characterize overlap across rounds $u < v$, define:

$$J_{i,u,v} = |C_{i,u} \cap C_{i,v}|, \quad J_{\max} = \max_{u < v} J_{i,u,v}$$

Because answers and evidence evolve over time, $J_{\max}$ tends to remain small for fast-changing topics.

Collision Probability. Sampling one QA pair per round, the probability that seed graph i produces the same QA pair twice within t rounds satisfies:

$$\Pr[\text{collision within } t] \leq \frac{t(t-1)}{2K^2} J_{\max} \quad (8)$$

Collision Control. To guarantee that this probability is at most $\delta \in (0, 1)$, it suffices to choose K such that:

$$K \geq \sqrt{\frac{t(t-1)}{2\delta} J_{\max}} \quad (9)$$

Equation (7) captures key trade-offs:

- Larger K (richer candidate pool) reduces collision probability for fixed t .
- Fewer evaluation rounds t allow a smaller K for the same error tolerance δ .
- Smaller $J_{\max}$ (i.e., greater semantic drift) reduces required K .

Because each seed graph is rebuilt from fresh evidence each round, its candidate set changes, helping keep $J_{\max}$ small. Equation (6) quantifies the collision risk, while Equation (7) provides a tunable design guideline for selecting K .

4.5 Cross-Round Reporting and Compatibility

Because each round rebuilds the seed graph set S_t , the underlying distribution $\mathcal{P}_t = \prod_i \mathcal{P}_{i,t}$ may drift over time, especially for evolving topics. Consequently, raw accuracies from different rounds may not be directly comparable. We offer two strategies:

i) Snapshot Evaluation. Fix a reference round t^* and freeze its seed graphs S_{t^*} . All systems are then evaluated on the same dataset D_{t^*} , ensuring identical test conditions.

ii) Macro-Averaged Score. For longitudinal tracking, aggregate performance across a window $\{t_1, \dots, t_T\}$ (e.g., most recent T rounds) using:

$$\text{Score}_{\text{macro}} = \frac{1}{T} \sum_{j=1}^T \text{Agg}(D_{t_j}, M) \quad (10)$$

Each D_{t_j} is i.i.d. from its own $\mathcal{P}_{t_j}$, so Eq. (8) summarizes average effectiveness under the benchmark’s natural evolution, without letting any single round dominate.

5 Results

6 Discussion

TBD

7 Conclusion

TBD

References

- [1] 2025. Searxng/Searxng. SearXNG.
- [2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. 2022. Improving Language Models by Retrieving from Trillions of Tokens. arXiv:2112.04426 [cs] doi:10.48550/arXiv.2112.04426 Comment: Fix incorrect reported numbers in Table 14.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models Are Few-Shot Learners. arXiv:2005.14165 [cs] doi:10.48550/arXiv.2005.14165 Comment: 40+32 pages.
- [4] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. 2023. Quantifying Memorization Across Neural Language Models. arXiv:2202.07646 [cs] doi:10.48550/arXiv.2202.07646
- [5] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. PaLM: Scaling Language Modeling with Pathways. arXiv:2204.02311 [cs] doi:10.48550/arXiv.2204.02311
- [6] Jesse Dodge, Maarten Sap, Ana Marasović, William Agnew, Gabriel Ilharco, Dirk Groeneveld, Margaret Mitchell, and Matt Gardner. 2021. Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus. arXiv:2104.08758 [cs] doi:10.48550/arXiv.2104.08758 Comment: EMNLP 2021 accepted paper camera ready version.
- [7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs] doi:10.48550/arXiv.2312.10997 Comment: Ongoing Work.
- [8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 3929–3938.
- [9] Ziwen Han, Meher Mankikar, Julian Michael, and Zifan Wang. 2025. Search-Time Data Contamination. arXiv:2508.13180 [cs] doi:10.48550/arXiv.2508.13180
- [10] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. arXiv:2007.01282 [cs] doi:10.48550/arXiv.2007.01282
- [11] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2022. Atlas: Few-shot Learning with Retrieval Augmented Language Models. arXiv:2208.03299 [cs] doi:10.48550/arXiv.2208.03299
- [12] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. arXiv:1705.03551 [cs] doi:10.48550/arXiv.1705.03551 Comment: Added

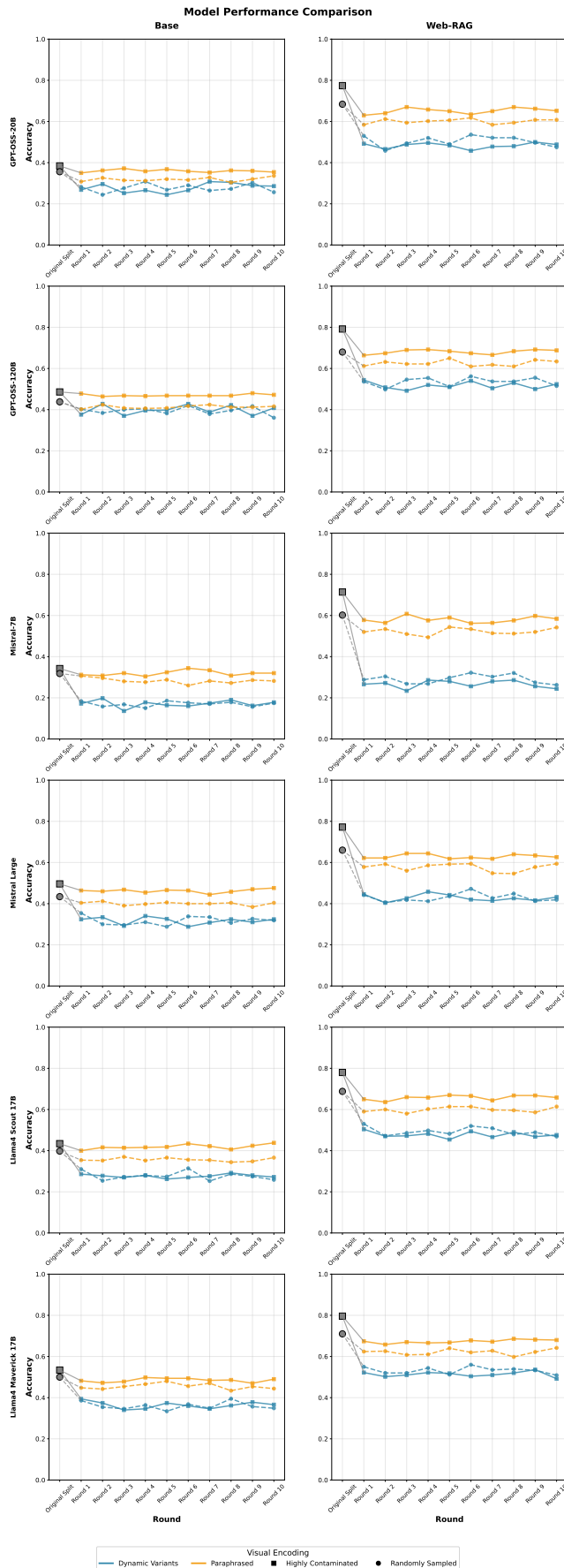


Figure 1: model_performance_collage.pdf

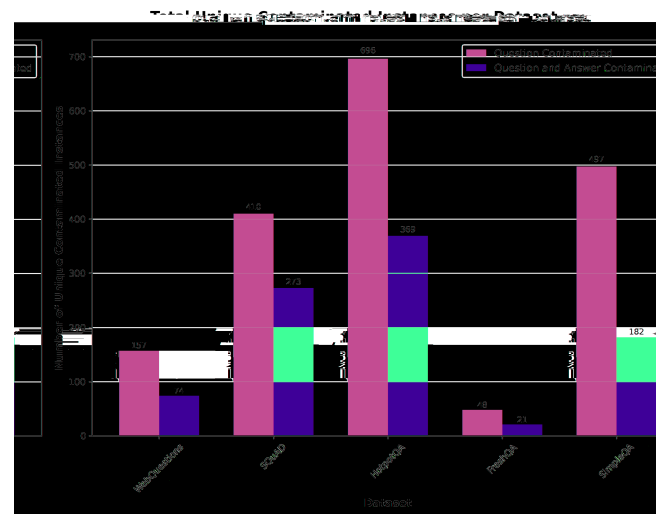


Figure 2: total_unique_contaminated_instances_per_dataset.png

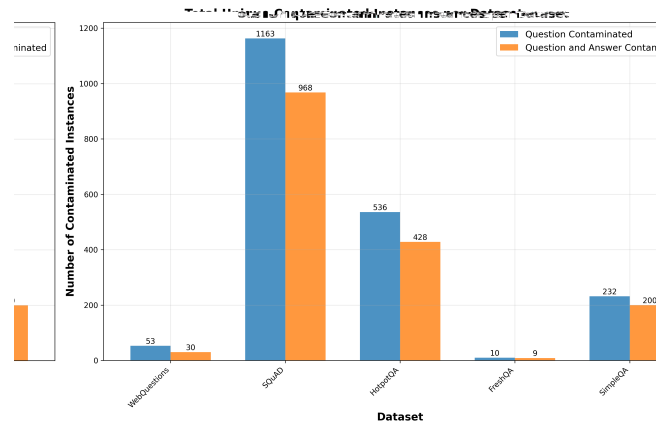


Figure 3: total_unique_retrieval_contaminated_instances_per_dataset.png

references, fixed typos, minor baseline update.

- [1] Jungo Kasai, Keisuke Sakaguchi, Yoichi Takahashi, Ronan Le Bras, Akari Asai, Xinyan Yu, Dragomir Radev, Noah A. Smith, Yejin Choi, and Kentaro Inui. 2024. RealTime QA: What's the Answer Right Now? arXiv:2207.13332 [cs] doi:10.48550/arXiv.2207.13332 Comment: RealTime QA Website: <https://realtimeqa.github.io/>.
- [2] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. 2021. Dynabench: Rethinking Benchmarking in NLP. arXiv:2104.14337 [cs] doi:10.48550/arXiv.2104.14337 Comment: NAACL 2021.
- [3] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tacl_a_00276
- [4] Ariel N. Lee, Cole J. Hunter, and Nataniel Ruiz. 2024. Platypus: Quick, Cheap, and Powerful Refinement of LLMs. arXiv:2308.07317 [cs] doi:10.48550/arXiv.2308.07317 Comment: Workshop on Instruction Tuning and Instruction Following at NeurIPS 2023.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented

- Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474.
- [] Yucheng Li, Frank Guerin, and Chenghua Lin. 2024. An Open Source Data Contamination Report for Large Language Models. arXiv:2310.17589 [cs] doi:10.48550/arXiv.2310.17589
- [] Ken Ziyu Liu, Christopher A. Choquette-Choo, Matthew Jagielski, Peter Kairouz, Sanmi Koyejo, Percy Liang, and Nicolas Papernot. 2025. Language Models May Verbatim Complete Text They Were Not Explicitly Trained On. arXiv:2503.17514 [cs] doi:10.48550/arXiv.2503.17514 Comment: Main text: 9 pages, 7 figures, 1 table. Appendix: 29 pages, 20 tables, 15 figures.
- [] Zhiyi Ma, Kawin Ethayarajh, Tristan Thrush, Somya Jain, Ledell Wu, Robin Jia, Christopher Potts, Adina Williams, and Douwe Kiela. 2021. Dynaboard: An Evaluation-As-A-Service Platform for Holistic Next-Generation Benchmarking. arXiv:2106.06052 [cs] doi:10.48550/arXiv.2106.06052
- [] Grégoire Mialon, Roberto Dessi, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. Augmented Language Models: A Survey. arXiv:2302.07842 [cs] doi:10.48550/arXiv.2302.07842
- [] John X. Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. 2025. How Much Do Language Models Memorize? arXiv:2505.24832 [cs] doi:10.48550/arXiv.2505.24832
- [] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2022. WebGPT: Browser-assisted Question-Answering with Human Feedback. arXiv:2112.09332 [cs] doi:10.48550/arXiv.2112.09332 Comment: 32 pages.
- [] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.